

Job Discovery Tool - Hiring Assignment

Background

GradGuide helps international students plan their study abroad journey, including finding work once they arrive: internships, full-time roles, and part-time or casual jobs like cafes and retail. We want to see how a candidate approaches building a job discovery tool for this audience.

The Assignment

Build a job discovery tool for international students to find part-time, full-time, and internship roles. Include everyday jobs (cafes, retail, delivery, tutoring), not just internships. The tool needs its own scraper (see below), a searchable and filterable listing interface, and 3 original features designed by the candidate.

Core Requirements

Area	Requirement
Job coverage	Part-time/casual, full-time, and internship roles
Data source	Real listings from their own scraper, no job APIs
Listing view	Title, employer, location, pay, job type, posted date

Add Your Own Features

Design and build the 3 best features you can think of for an international student job search. No fixed list, we want to see your own judgment. Ideas to spark thinking:

- Visa or work-hour awareness
- Commute or campus proximity matching
- Employer trust signals
- Application tracking or reminders
- Language comfort filters

Include a short rationale for each: what problem it solves and why it matters for this audience.

The Scraper Requirement

The candidate must build their **own** scraper to source job listings. This is a hard requirement, not a nice-to-have.

- No paid or free job-board APIs (Indeed API, LinkedIn Jobs API, Adzuna, etc.)
- No AI/LLM-based scraping tools or services that do the extraction for them (no "AI scraper" SaaS products, no prompting an AI to fetch and structure the data on their behalf)

- The scraper itself should be their own code: fetching pages, parsing HTML, and structuring the data into listings
- It's fine to scrape a small, defined set of real sites (e.g. a couple of local job boards, café/retail chain career pages, campus job boards) rather than the entire internet. Breadth matters less than the scraper actually being theirs and working
- They should be able to explain how it works and defend their design choices in a follow-up conversation

Deliverables

1. Working app (hosted link or run instructions)
2. Source code (repo link)
3. Short write-up (1 page): how the scraper works, rationale for the 3 features
4. Add a short video walkthrough to the GitHub README explaining the solution, key features, and implementation.

Evaluation Criteria

Criterion	What we're looking for
Scraper quality	Works, is their own code, handles real HTML
Job coverage	Doesn't skip part-time/casual jobs
Feature judgment	Real understanding of student problems, not generic filters
Usability	Clear, fast, usable
Code quality	Readable, reasonably structured
Communication	Can explain decisions clearly